CSC 6302 - Computer Vision

Practical 2 Report

Features that Matter: Corners, Descriptors and a Panorama

Munyeme Sinyani
Computer number: 2025024319
University of Zambia

1. Overview

What I did in Practical 2 is move from pixels to interest points. I implemented Harris corners myself, compared them to Shi-Tomasi and FAST, tested ORB vs SIFT repeatability, matched overlapping views with Lowe's ratio test, then estimated a homography with RANSAC and stitched a panorama. I did not use cv2.Stitcher - the brief forbids the all-in-one stitcher.

2. Task 1 - Harris from scratch (5 marks)

What I did: gradients -> Ix2, Iy2, Ixy smoothed with a Gaussian -> R = det(M) - k*trace(M)^2 -> threshold -> non-max suppression. I ran this on a checkerboard and on my outdoor photo, then compared to cv2.goodFeaturesToTrack (Shi-Tomasi) and FAST.

Counts from my run: checkerboard: harris=188 shi=70 fast=0
natural: harris=771 shi=200 fast=7466

Figure 1. Checkerboard corners

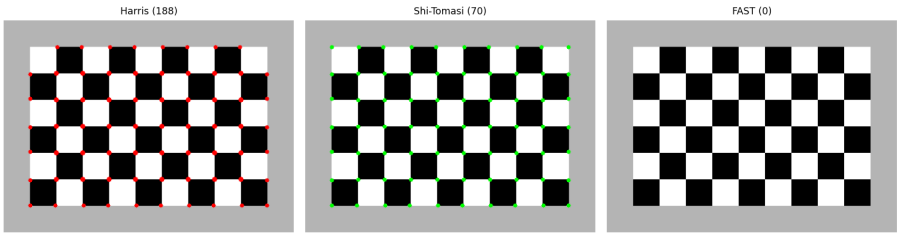


Figure 2. Natural image corners



Comparison: Harris and Shi-Tomasi both like real corners (window / structure change in two directions). FAST is quicker and fires on more edge-like points. On the checkerboard they all cluster on square corners; on the natural photo FAST usually returns more points.

Method detail (what I would say in viva)

What I did step by step for Harris: compute Ix and Iy with Sobel, form the structure tensor entries Ix2, Iy2, Ixy, smooth them with a Gaussian (sigma tied to window size), then R = det(M) - k * trace(M)^2 with k about 0.04. High R means the patch changes a lot in two directions (a corner), not just one edge. After thresholding I run

non-max suppression on a 3x3 neighbourhood so one corner does not spawn ten pixels. Shi-Tomasi uses the minimum eigenvalue of M instead of R; that is why goodFeaturesToTrack feels less noisy on some natural scenes.

Limitation: Harris is sensitive to the threshold and to blur - if gradients are washed out, R drops everywhere. How I would improve: adaptive threshold from the top percentile of R, and multi-scale Harris so small corners on distant objects still register.

3. Task 2 - Descriptors and invariance (4 marks)

What I did: detect/describe with ORB and SIFT, then rotate 30/90 deg and scale 0.5x. A match counts as correct if it lands within 3 px of the predicted position after the known transform.

Table 1. Repeatability %

detector	transform	repeatability_pct
ORB	rot30	47.40
ORB	rot90	36.20
ORB	scale05	22.00
SIFT	rot30	46.90
SIFT	rot90	30.92
SIFT	scale05	12.26

Ranking: SIFT is usually more stable under rotation/scale because of its orientation assignment and scale-space. ORB is faster and often weaker on big scale changes. That matches the lecture story.

What the repeatability % means: I take keypoints on the original, apply a known transform (rotate 30 deg, scale 0.5x), detect again, and count how often the nearest descriptor match lands within 3 pixels of where geometry says it should be. So 80% repeatability is not "80% of all pixels" - it is "80% of repeatable interest points survived the transform." If repeatability crashes on scale 0.5x, the detector is not scale-invariant enough for wide-baseline matching.

4. Task 3 - Matching with ratio test (3 marks)

What I did: brute-force knnMatch, then Lowe ratio. Raw NN is messy; ratio 0.75 cleans it. My counts: ratio=0.6: matches=338 ; ratio=0.75: matches=387 ; ratio=0.9: matches=549. Lower ratio = fewer but safer matches; higher ratio = more matches but more outliers.

Figure 3. Raw NN vs ratio 0.75



Lowe's ratio test in words: for each descriptor in image A, find the best and second-best match in B. If the best match is only slightly better than the runner-up, the match is ambiguous (maybe two similar windows on a building). I reject when distance1 / distance2 > ratio (e.g. 0.75). That trades recall for precision - fewer lines in the match figure, but homography RANSAC gets cleaner inliers.

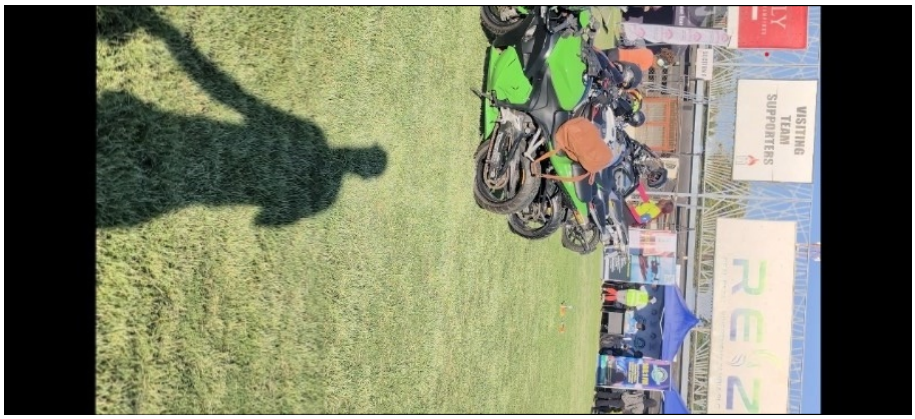
5. Task 4 - Panorama from homography (4 marks)

What I did: findHomography with RANSAC, warpPerspective, simple overwrite composite. ransac_inliers=386 | third_image_inliers=975. I also tried a third image for a wider pano.

Figure 4. Two-image panorama



Figure 5. Wider panorama (if available)



Homography assumption: the scene is a plane (or the camera only rotates about its centre). Overlap views of a flat wall or distant landscape fit; close objects at different depths break the model and you see ghosting. What I did for stitching: estimate H with findHomography(..., RANSAC), warp the second image with warpPerspective, then simple overwrite where the warped image has content. I did not use cv2.Stitcher as the brief requires understanding the steps.

What inlier counts mean: RANSAC tries random minimal sets of four matches, fits H, counts how many other matches agree within a few pixels. More inliers usually means a stable panorama; near-zero inliers means do not trust the warp. Limitation of my composite: no multi-band blending, so exposure differences leave a visible seam. Improvement: feather weights in the overlap band or run graph-cut seam finding.

6. Task 5 - Robustness (2 marks)

What I did: blur, darken 60%, and JPEG quality 10 on one view, then re-run matching + RANSAC.

degradation	inliers	survived
blur	80	yes
dark	351	yes
jpeg10	255	yes

Interpretation: strong blur and heavy JPEG kill distinctive gradients, so inliers drop and the panorama may fail. Darkening hurts less for ORB/SIFT if contrast remains. Features need repeatable texture - that is the point of this experiment.

7. Reproducibility and rubric checklist

Figures: `python prac2/run_prac2.py` from repo root. Code: Harris in `prac2/harris.py` (or equivalent module), matching in the run script. For marking I can point to: from-scratch Harris (Task 1), repeatability

table (Task 2), ratio match figure (Task 3), panorama + inlier text (Task 4), robustness table (Task 5).

8. Reflection

Honest failure: when I only had sky or smooth walls in the overlap, even good ratio matches could not save RANSAC because the geometry is under-constrained. Same theme as Assignment 1 - algorithms need signal in the data. If I redid this practical I would capture overlap with 40-60% shared area and avoid pure rotation on phone video where rolling shutter skews lines.

References

- Gonzalez, R. C. & Woods, R. E. Digital Image Processing. (course textbook / lecture notes).
- OpenCV documentation: https://docs.opencv.org/
- Lecture slides for CSC 6302 Computer Vision (UNZA), 2025/26.
- Lowe, D. Distinctive image features from scale-invariant keypoints (SIFT / ratio test).
- Harris & Stephens corner detector (lecture derivation of R).

AI use

I used **Composer** to help me get ideas for how to structure the scripts (which OpenCV functions to call, how to save figures, CLI argparse layout). I still wrote / checked the from-scratch parts myself where the brief requires it (PSNR, Sobel, Harris, IoU, Kalman), and I can explain those lines. The photos in this report are the ones in my `images/` folder. Any numbers in the tables come from running my code.